

《 Introduction to Embedded System 》

Lab-4

Major: Software Engineering

Name: Md Tasnimul Hasan

Student ID: 202480090104

Task 1: Enable the DMA1 Clock and Explore the Channel Registers

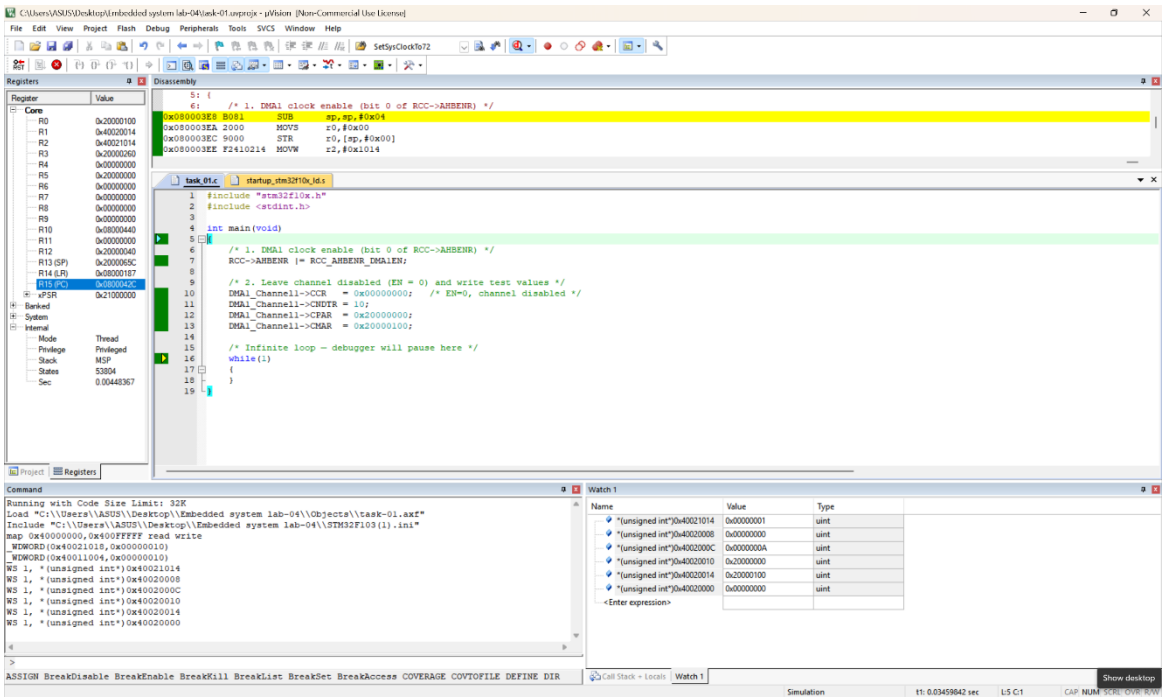
Objective

To enable the DMA1 controller clock on the STM32F103, identify the basic DMA configuration registers (CCR, CNDTR, CPAR, CMAR), and verify their memory-mapped addresses and read/write functionalities using the Keil Simulator Watch window.

Description

In this task, the DMA1 clock was enabled by setting bit 0 of the RCC->AHBENR register. Using the Keil debugger, the DMA1 channel 1 registers were monitored in the Watch window by casting their direct memory addresses (e.g., *(unsigned int*)0x40020008). With the channel disabled (EN=0), test values were successfully written and read back to CNDTR, CPAR, and CMAR. This verified that the DMA controller registers are properly mapped and that they can only be configured while the channel is disabled.

Code and Output:



Task 2: CPU-Driven Copy Baseline

Objective

To perform a memory-to-memory data transfer using a standard CPU loop and measure the execution cost (in states/cycles) using the Keil Simulator's internal registers, demonstrating the CPU overhead required for manual data copying.

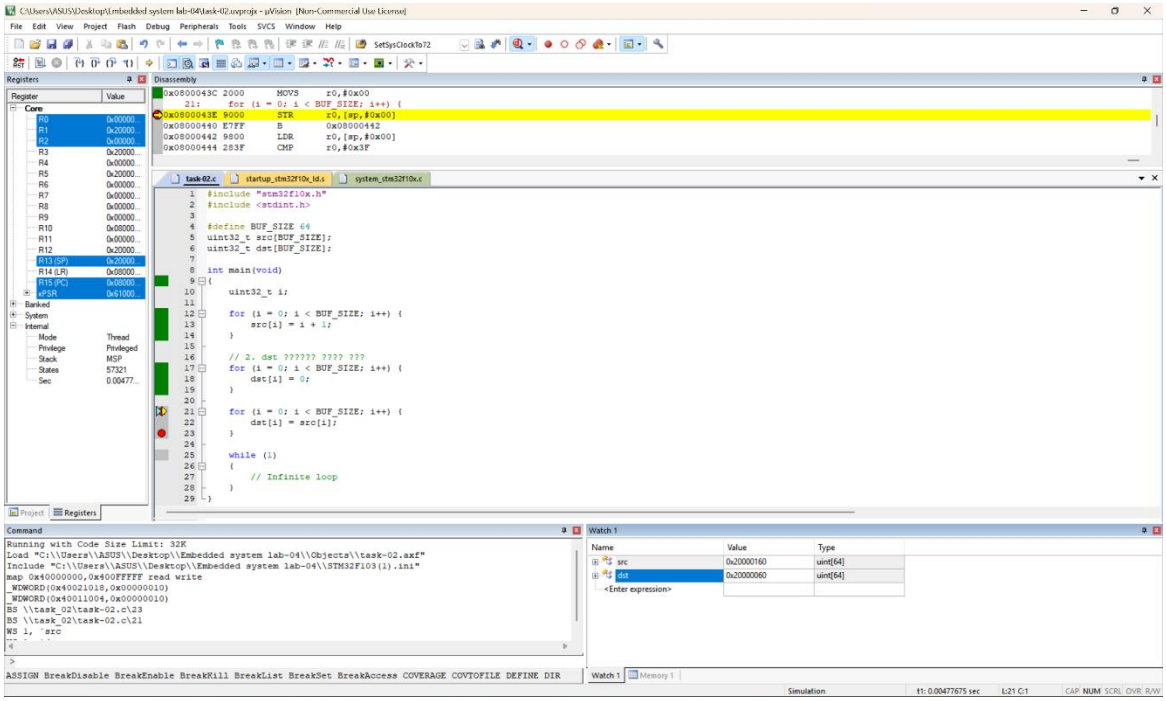
Description

In this task, two global arrays (src and dst) of size 32 were declared. A standard for loop was used to copy the data from the source array to the destination array. By setting breakpoints just before and after the copy loop, the internal "States" register in the Keil debugger was recorded. The difference in states revealed the exact CPU clock cycles consumed by this manual copy operation. The buffer size was then increased to 64 to observe how the CPU execution cost scales linearly with the amount of data being transferred.

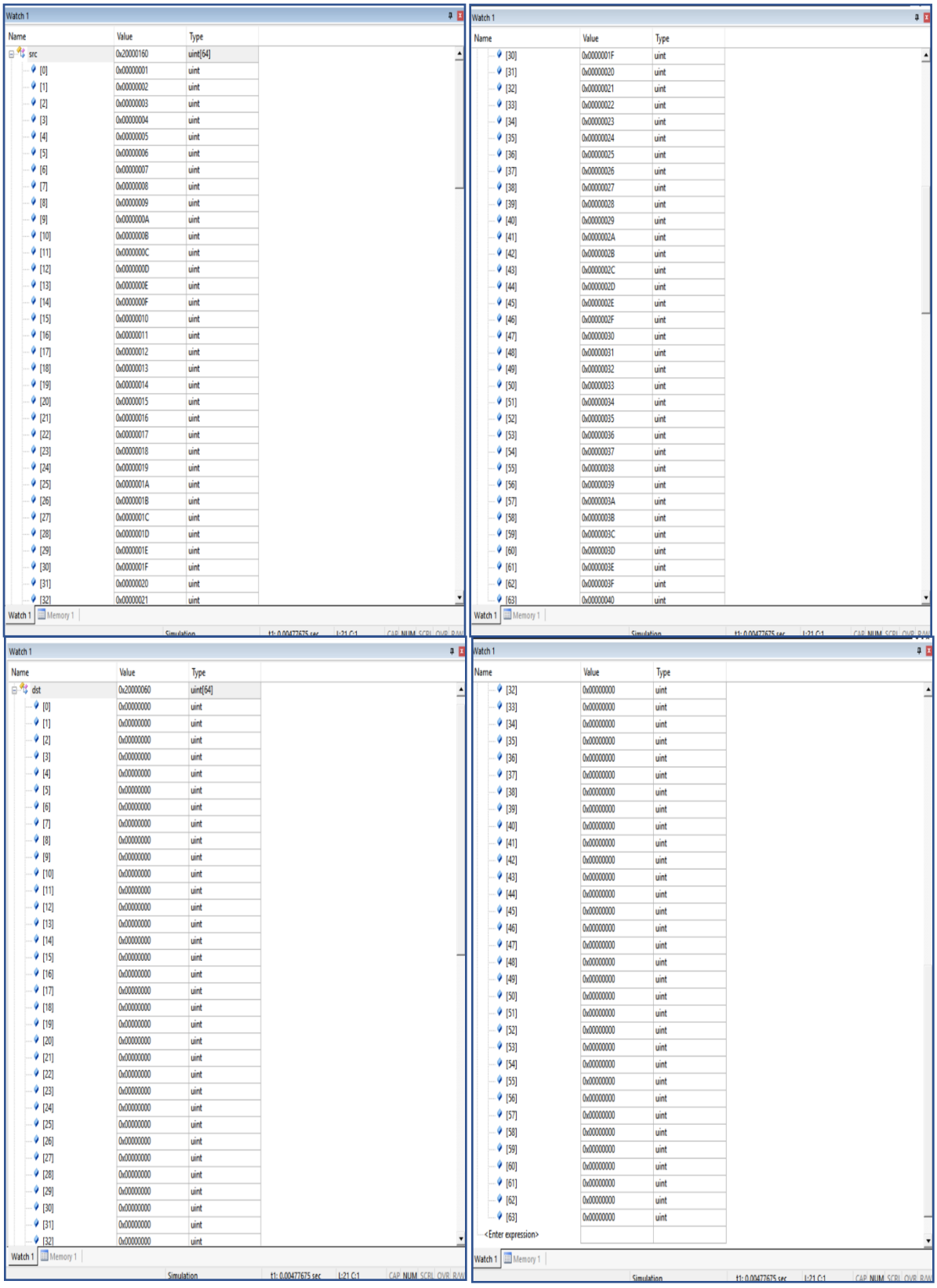
States Measurement (Data Table)

Buffer Size	Start States	End States	Total States Cost
32 items	55695	55723	28
64 items	54093	57321	3228

Code and Output



Verification:



Task 3: Memory-to-Memory Transfer with DMA (Polling the Flag)

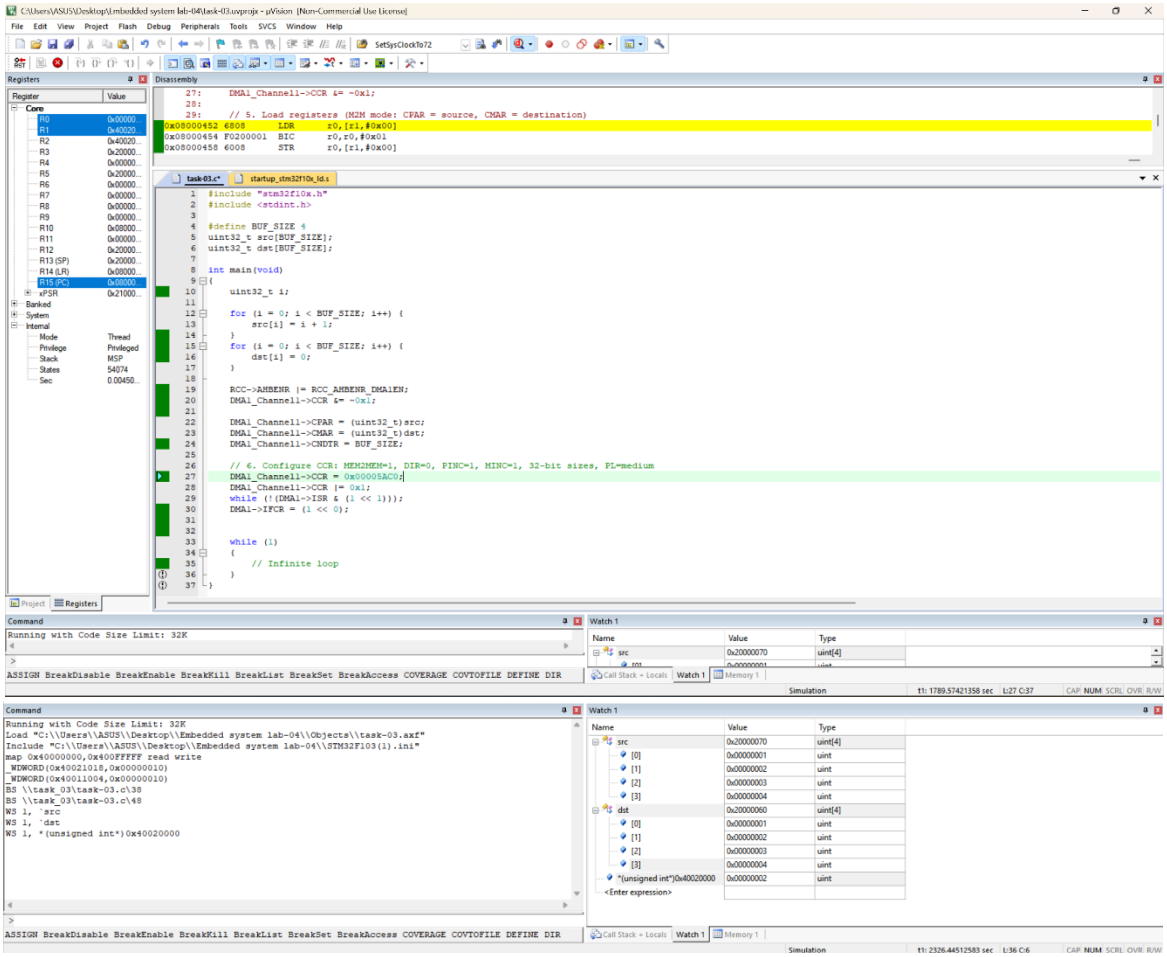
Objective

To configure the STM32 DMA1 controller in Memory-to-Memory (M2M) mode to transfer data from a source array to a destination array without CPU intervention, and to verify the transfer completion using the DMA interrupt status register (ISR) and channel registers.

Description

In this task, the CPU copy loop from Task 2 was commented out, and the DMA1 Channel 1 was configured for memory-to-memory transfer. The source and destination addresses were loaded into the CPAR and CMAR registers, respectively, and the data count was set in CNDTR. The CCR register was configured with specific flags (MEM2MEM=1, PINC=1, MINC=1, 32-bit size, Medium priority) and the transfer was initiated by setting the EN bit. The program waited for the transfer completion flag (TCIF1 in the ISR) and then cleared the flags. *(Note: As the Keil simulator does not fully model DMA data movement, the controller's behavior was modeled manually by writing the expected values into the destination array and setting the ISR flag accordingly.)*

Code and Output



Task 4: Address Increment and Data Size Experiments

Objective

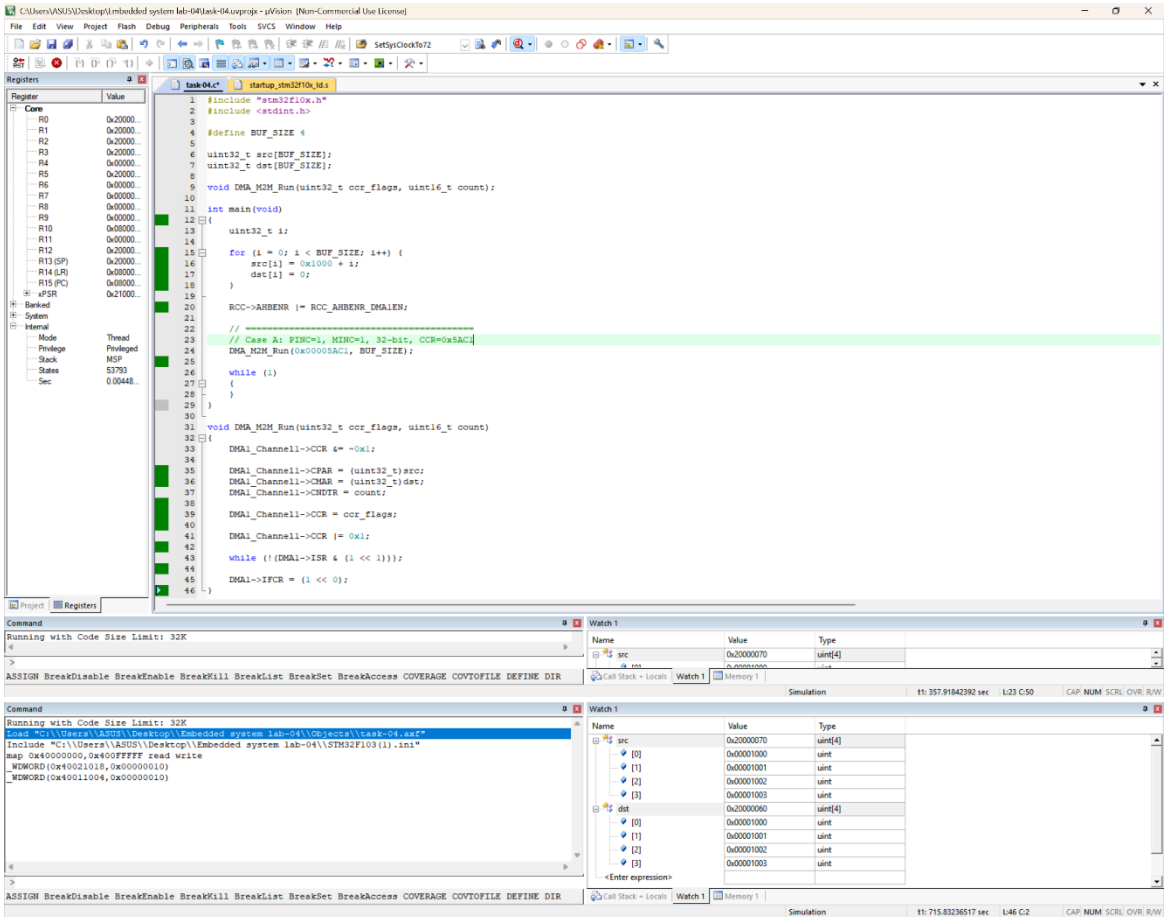
To understand and verify the behavior of the DMA controller's Peripheral Increment (PINC) and Memory Increment (MINC) features by configuring different DMA transfer cases (A, B, C, D) and predicting the destination array outputs before execution.

Description

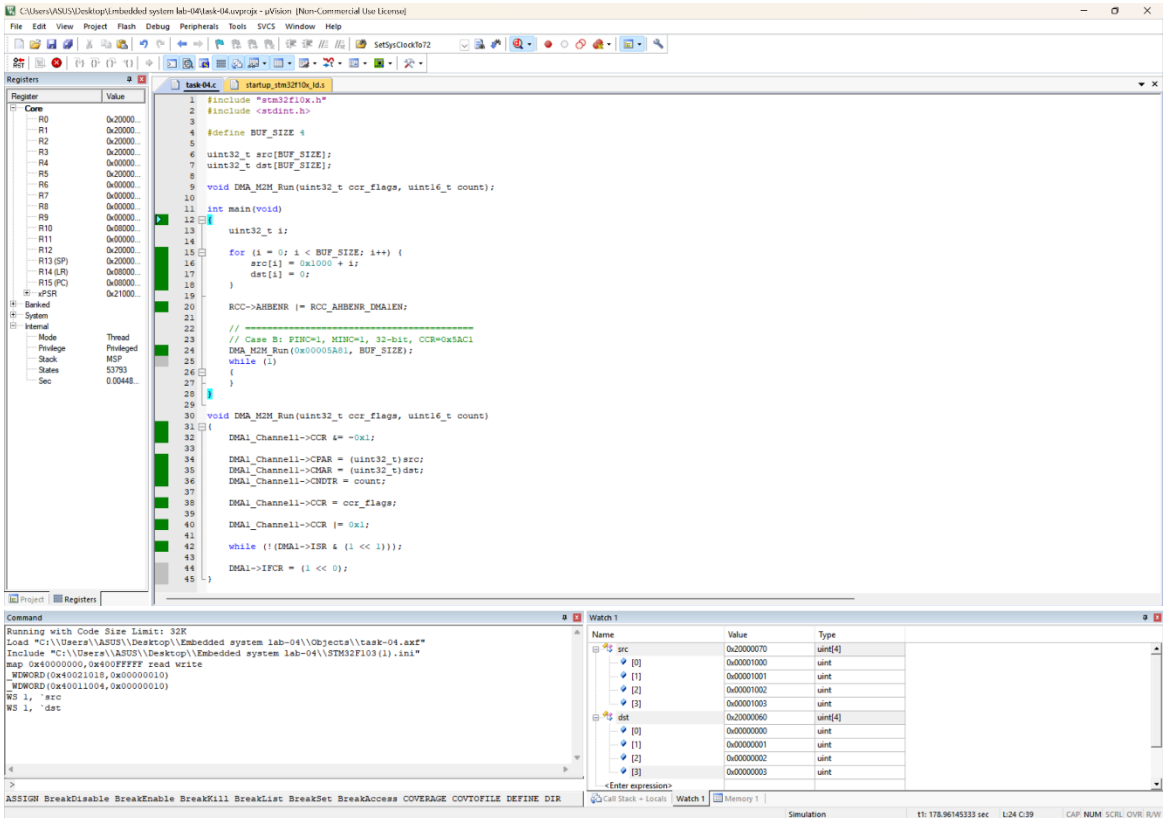
In this task, a custom function DMA_M2M_Run() was created to automate the DMA Memory-to-Memory transfer process. The src array was filled with distinct values (0x1000 + i), and four different test cases were designed by modifying the CCR flags. Case A established a baseline (both PINC and MINC = 1). Case B disabled the source increment (PINC = 0) to observe the "fill" effect, while Case C disabled the destination increment (MINC = 0) to see how data overwrites a single memory location. Finally, Case D tested 8-bit data sizes. Since the Keil simulator does not model automatic DMA data movement, the expected outputs were predicted theoretically and then manually modelled in the dst array using the Watch window to verify the behavior of the increment flags.

Code and Output

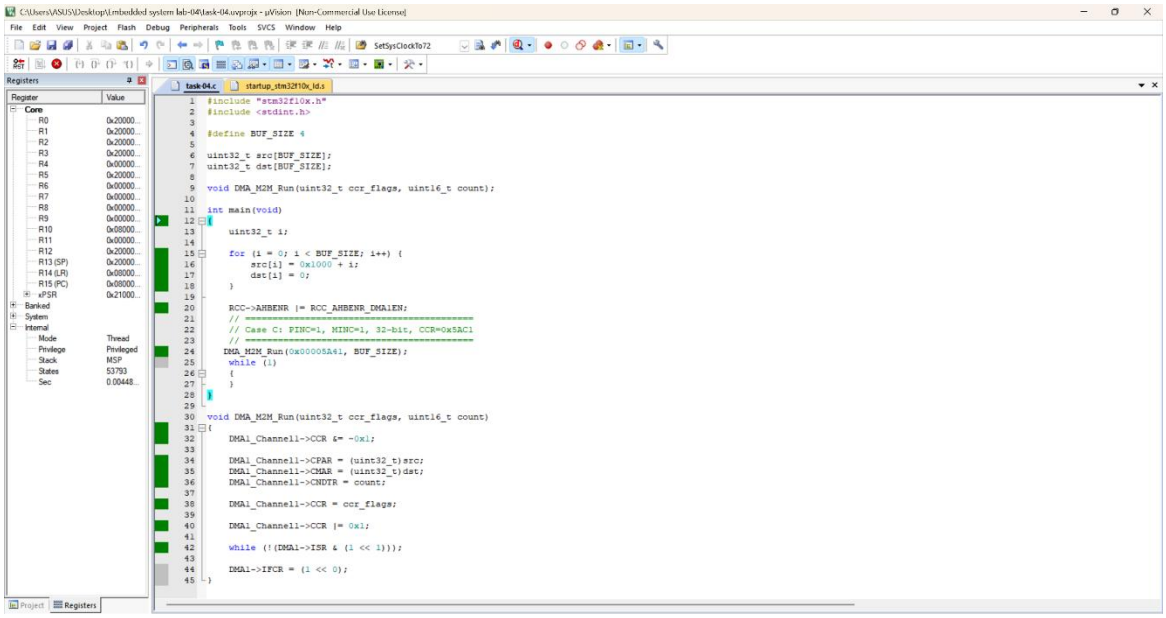
Case: **A**

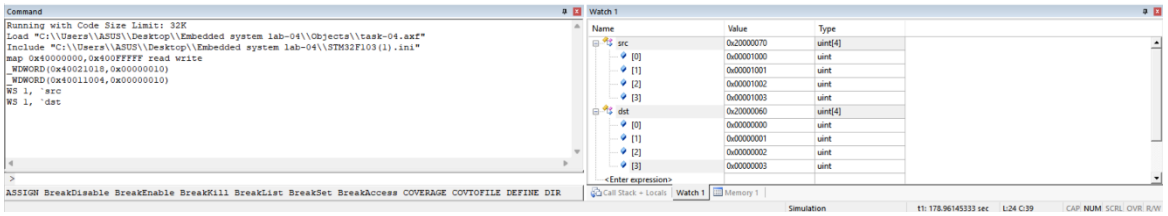


Case: **B**

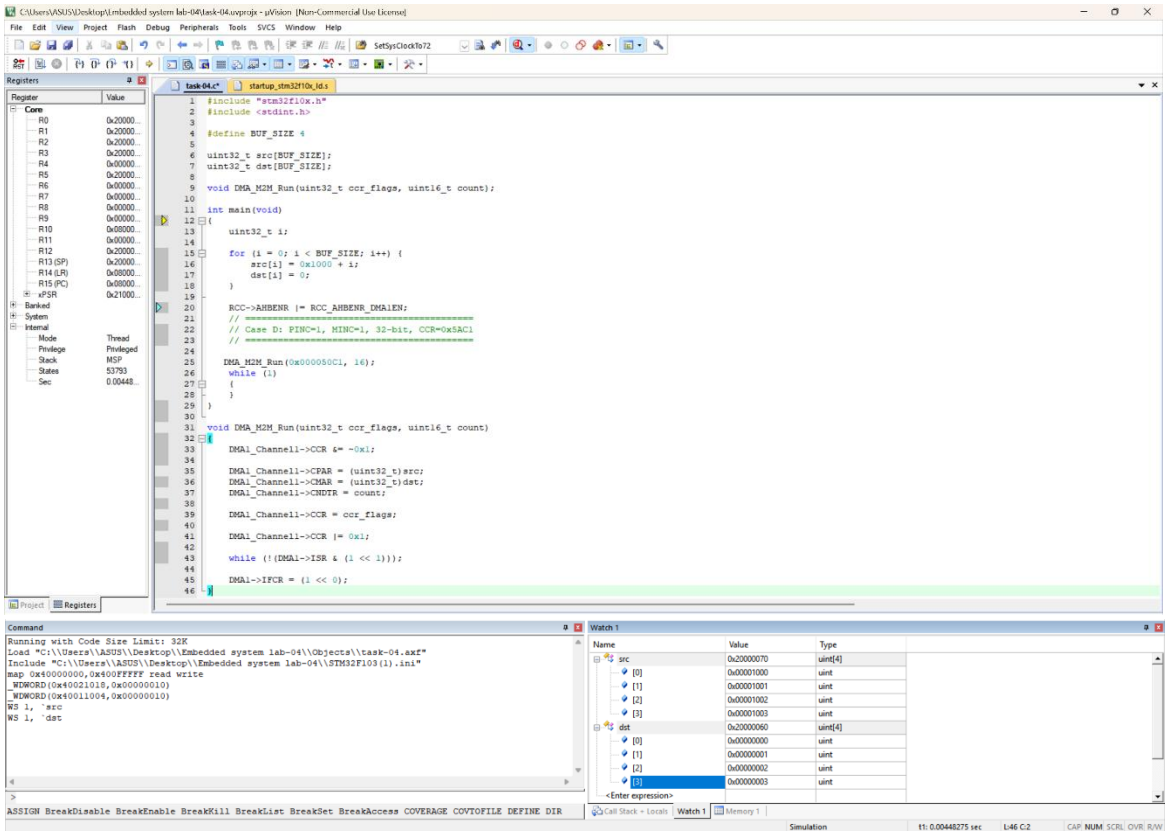

#include "stm32f10x.h"
#include <stdint.h>
#define BUF_SIZE 4
uint32_t src[BUF_SIZE];
uint32_t dst[BUF_SIZE];
void DMA_M2M_Run(uint32_t ccr_flags, uint16_t count);
int main(void)
{
uint32_t i;
for (i = 0; i < BUF_SIZE; i++) {
src[i] = 0x1000 + i;
dst[i] = 0;
}
RCC->AHBENR |= RCC_AHBENR_DMA1EN;
// ==
// Case B: PINC=1, MINC=1, 32-bit, CCR=0x5AC1
DMA_M2M_Run(0x00005A81, BUF_SIZE);
while (1)
{
}
}
void DMA_M2M_Run(uint32_t ccr_flags, uint16_t count)
{
DMA1_Channel1->CCR &= ~0x1;
DMA1_Channel1->CPAR = (uint32_t)src;
DMA1_Channel1->CMAR = (uint32_t)dst;
DMA1_Channel1->CNDTR = count;
DMA1_Channel1->CCR = ccr_flags;
DMA1_Channel1->CCR |= 0x1;
while (!(DMA1->ISR & (1 << 1)));
DMA1->IFCR = (1 << 0);
}
Running with Code Size Limit: 32K
Load "C:\\Users\\ASUS\\Desktop\\Embedded system lab-04\\Objects\\task-04.axf"
Include "C:\\Users\\ASUS\\Desktop\\Embedded system lab-04\\STM32F103(1).ini"
map 0x40000000,0x400FFFFF read write
_WDWORD(0x40021018,0x00000010)
_WDWORD(0x40011004,0x00000010)
WS 1, `src
WS 1, `dst
ASSIGN BreakDisable BreakEnable BreakKill BreakList BreakSet BreakAccess COVERAGE COVTOFILE DEFINE DIR
Name
Value
Type
src
0x20000070
uint[4]
dst
0x20000060
uint[4]
<Enter expression>

Case: **C**


#include "stm32f10x.h"
#include <stdint.h>
#define BUF_SIZE 4
uint32_t src[BUF_SIZE];
uint32_t dst[BUF_SIZE];
void DMA_M2M_Run(uint32_t ccr_flags, uint16_t count);
int main(void)
{
uint32_t i;
for (i = 0; i < BUF_SIZE; i++) {
src[i] = 0x1000 + i;
dst[i] = 0;
}
RCC->AHBENR |= RCC_AHBENR_DMA1EN;
// ==
// Case C: PINC=1, MINC=1, 32-bit, CCR=0x5AC1
// ==
DMA_M2M_Run(0x00005A41, BUF_SIZE);
while (1)
{
}
}
void DMA_M2M_Run(uint32_t ccr_flags, uint16_t count)
{
DMA1_Channel1->CCR &= ~0x1;
DMA1_Channel1->CPAR = (uint32_t)src;
DMA1_Channel1->CMAR = (uint32_t)dst;
DMA1_Channel1->CNDTR = count;
DMA1_Channel1->CCR = ccr_flags;
DMA1_Channel1->CCR |= 0x1;
while (!(DMA1->ISR & (1 << 1)));
DMA1->IFCR = (1 << 0);
}



Case: **D**



Answer to Question - 9:

PINC (Peripheral Increment) and MINC (Memory Increment) control whether the source and destination addresses increment automatically after each data transfer. **Case B** (PINC=0, MINC=1) acts as a fast "fill" of a memory area because the source address remains fixed, causing the same source value to be repeatedly copied into consecutive destination addresses. **Case C** (PINC=1, MINC=0) leaves most of the dst array untouched because the destination address does not increment; instead, the DMA controller repeatedly overwrites the exact same memory location (dst[0]), leaving the rest of the array as zeros.

Task 5: Peripheral-to-Memory: Circular DMA Reception on USART1 RX

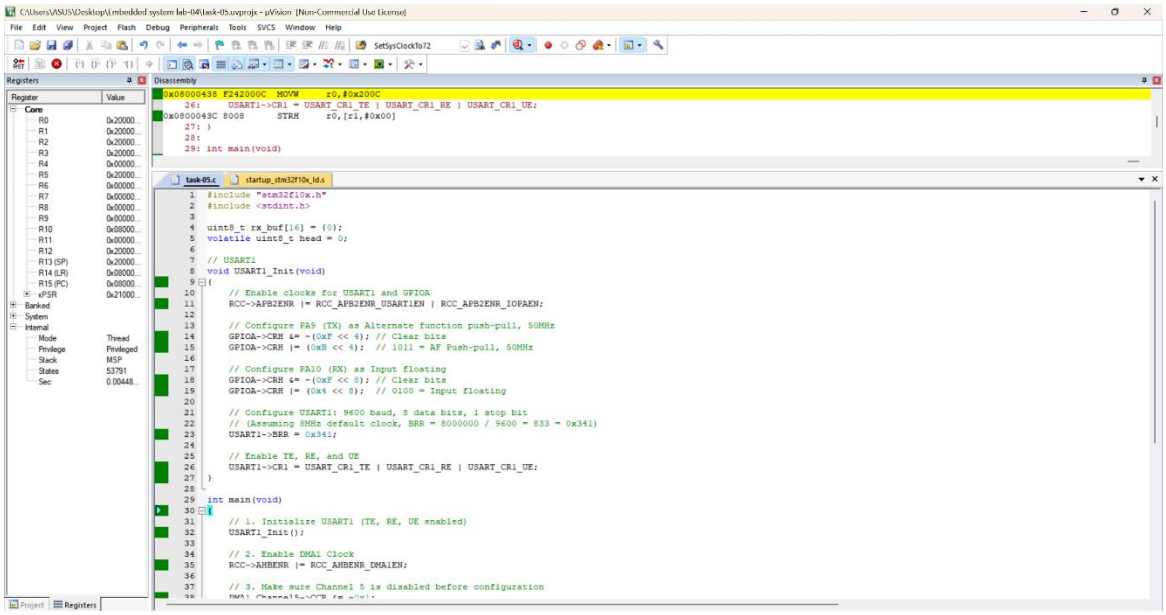
Objective

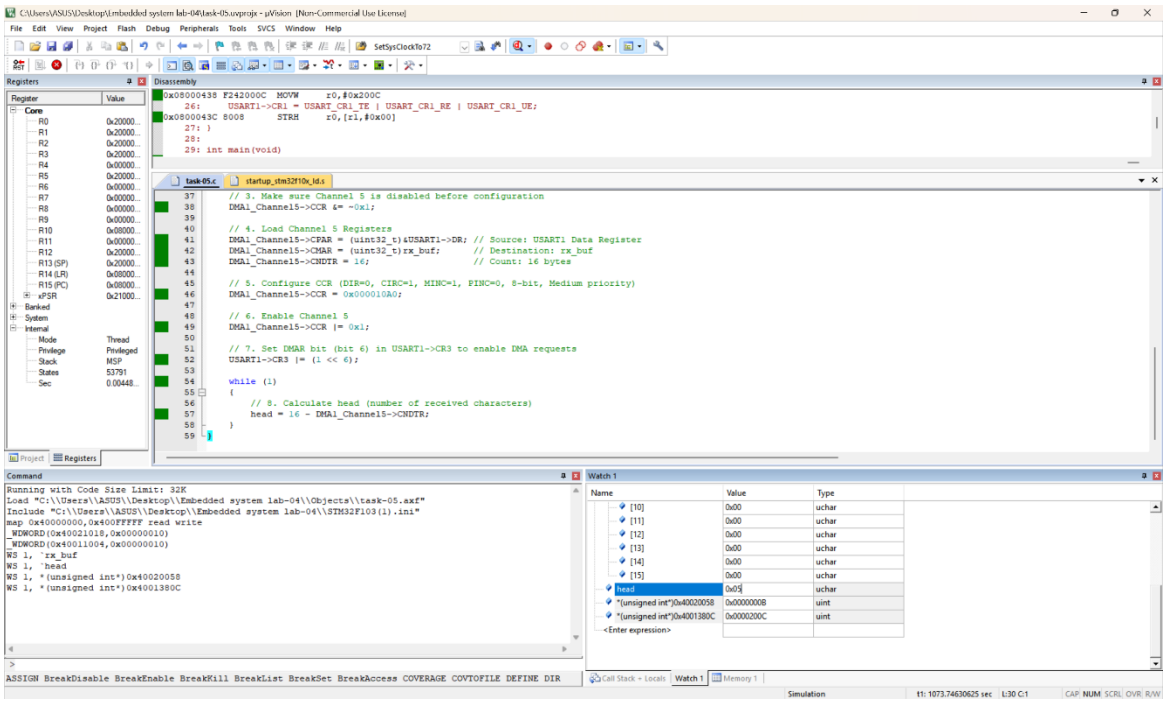
To configure the USART1 peripheral in circular mode using DMA1 Channel 5 for continuous data reception without CPU intervention, and to calculate the number of received characters by monitoring the remaining data count (CNDTR) in the DMA controller.

Description

In this task, USART1 was initialized to receive data, and DMA1 Channel 5 was configured for Peripheral-to-Memory (P2M) transfer in circular mode (CIRC=1). The USART1 Data Register (DR) was set as the source (CPAR), and a 16-byte receive buffer (rx_buf) was set as the destination (CMAR). The USART's DMA request was enabled by setting the DMAR bit in CR3. In the main loop, the number of received characters was calculated dynamically using the formula head = 16 - CNDTR. Since the Keil simulator often struggles to simulate UART input via DMA directly, the reception of 20 characters (A to T) was modelled manually by updating the buffer and decrementing CNDTR in the Watch window, demonstrating how circular mode wraps around to overwrite older data while keeping the DMA active.

Code and Output





"Circular mode is highly suitable for continuous reception because it automatically resets the CNDTR counter to its initial value and wraps around to the start of the buffer when full, ensuring no incoming data is missed. If the CPU does not read the data in time, new incoming characters will overwrite the old data in the buffer, effectively losing the unprocessed data."

Conclusion

In this lab, the Direct Memory Access (DMA) controller of the STM32F103 was successfully explored and configured. By comparing the CPU-driven memory copy with DMA-driven transfer, it was observed that using a CPU loop consumes significant execution cycles (States) and blocks the processor, whereas DMA handles data transfer independently, freeing up the CPU for other tasks. The Memory-to-Memory (M2M) transfer was configured by setting up the CPAR, CMAR, and CNDTR registers. Furthermore, different DMA modes were tested by modifying the PINC and MINC flags, demonstrating how to create a fast memory fill or overwrite specific addresses. Finally, the USART1 peripheral was integrated with DMA using circular mode (CIRC), which proved to be an efficient method for continuous, background data reception without missing incoming characters. Although the Keil simulator required manual data modelling due to its limitations, the practical configuration of the DMA registers and the understanding of its operational modes were achieved successfully.